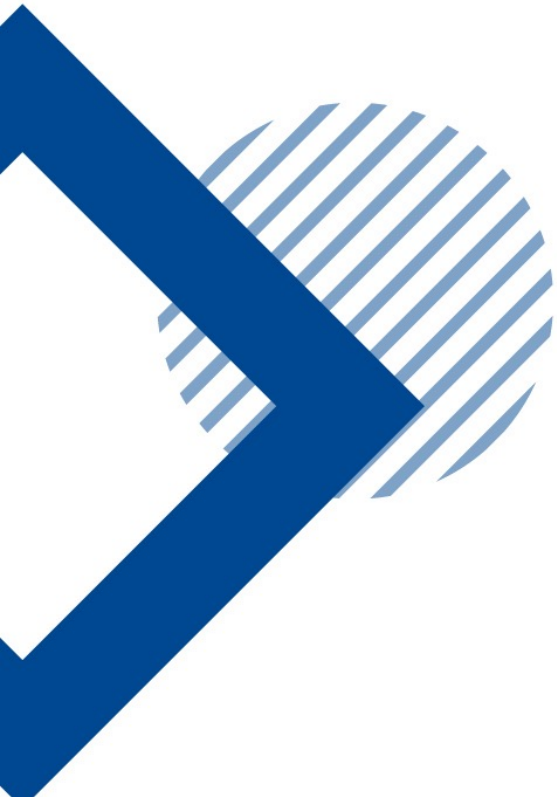
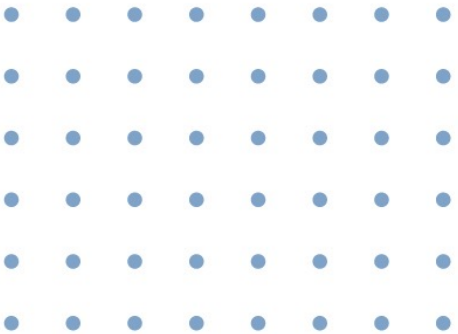
Персистентность

Интернет программирование







2022

Персистентность

Персистентность (Persistence) – способность программы сохранять данные между запусками процесса (долговременное хранение состояния).

Без персистентности программа может хранить данные только в оперативной памяти, что не позволяет сохранять состояние и данные процесса при выключении компьютера.

Самый простой выход – сохранять состояние программы в файле при завершении с последующим восстановлением при запуске.

При использовании файла будут сложности при увеличении количества данных.

Решение – использовать базы данных.

ООП и реляционные базы данных

ООП и реляционные базы данных (РБД) – самые распространенные подходы к построению приложений с персистентностью.

В ООП данные представляют собой множество связанных объектов некоторых классов.

В РБД данные представляют собой множества связанных таблиц.

Для сопоставления объектов и таблиц используются механизмы объектно-реляционного отображения (ORM, Object-Relational Mapping).

Для работы с РБД используются механизмы управления персистентностью: соединение с РБД, управление транзакциями, кэширование, выполнение запросов к РБД, и т. д.

Сущность

Сущность – основная единица персистентности.

Сущности описывают некоторые объекты предметной области и атрибуты, значимые для решения задачи.

В ООП сущности представляются классами, их полями и методами.

В РБД сущности представлены таблицами, их атрибутами и связями с другими таблицами.
РБД хранит только состояние сущностей.

Выделение сущностей – основная задача при проектировании программы и хранилища данных.

Механизмы ORM и персистентность работают на уровне сущностей.

Java Persistence Api

Java Persistence Api (JPA) – спецификация, описывающая механизмы работы с реляционными данными в Java приложениях. JPA не реализует данные механизмы, а предоставляет набор интерфейсов для дальнейшей реализации.

Самая популярная реализация JPA – фреймворк Hibernate.

Основные компоненты фреймворка Hibernate:

- Hibernate ORM
- Hibernate OGM
- Hibernate EntityManager
- Hibernate Validator

Spring Data JPA = Spring + Hibernate

Spring Data JPA – часть проекта Spring Data для реализации персистентности в РБД.

Фреймворк Spring Data JPA основан на фреймворке Hibernate и предоставляет дополнительные абстракции и функциональные возможности.

Примеры доступны по ссылке: https://github.com/spring-projects/spring-data-examples/tree/main/jpa

ORM – Таблицы

Аннотация @Table

```
@Entity
@Table(name = "tab_students")
public class Student {
...
}
```

Аннотация @SecondaryTable

```
@Entity
@SecondaryTables({
    @SecondaryTable(name = "student"),
    @SecondaryTable(name = "address")
})
public class Student {
    @Column(table = "student")
    private String firstName;
    @Column(table = "student")
    private String lastName;
   @Column(table = "address")
   private String city;
   @Column(table = "address")
   private String street;
}
```

ORM – Первичные ключи

Аннотации @Id и @GeneratedValue

```
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    ...
}
```

GenerationType:

- TABLE – таблица с последовательностями первичных ключей,
- SEQUENCE – последовательность,
- IDENTITY – столбец идентификаторов,
- AUTO – автоматический выбор.

ORM – Составные первичные ключи – 1

Аннотации @Embeddable и @EmbeddedId

```
@Embeddable
public class StudentId {
    private String docNum;
    private String group;
    ...
}

@Entity
public class Student {
    @EmbeddedId
    private StudentId id;
    private String firstName;
    private String lastName;
    ...
}
```

ORM – Составные первичные ключи – 2

Аннотация @IdClass

```
public class StudentId {
    private String docNum;
    private String group;
    ...
}

@Entity
@IdClass(StudentId.class)
public class Student {
    @Id
    private String docNum;
    @Id
    private String group;
    private String firstName;
    private String lastName;
    ...
}
```

ORM – Атрибуты – 1

Аннотация @Basic

```
@Entity
public class Student {
    ...
    @Basic(fetch = FetchType.LAZY)
    @Lob
    private byte[] photo;
    ...
}
```

Аннотация @Column

```
@Entity
public class Student {
     ...
    @Column(nullable = false, updatable = false)
    private String firstName;
    @Column(nullable = false)
    private String lastName;
    ...
}
```

ORM – Атрибуты – 2

Аннотация @Temporal

```
@Entity
public class Student {
    ...
    @Temporal(TemporalType.DATE)
    private Date dateOfBirth;
    @Temporal(TemporalType.TIMESTAMP)
    private Date creationDate;
    ...
}
```

Аннотация @Transient

```
@Entity
public class Student {
     ...
    @Transient
     private Integer age;
    ...
}
```

ORM – Атрибуты – 3

Аннотация @Enumerated

```
public enum StudentType {
    BUDGET,
    VNEBUDGET
}

@Entity
public class Student {
    ...
    @Enumerated(EnumType.STRING)
    private StudentType studentType;
    ...
}
```

ORM – Типы доступа к атрибутам

Аннотация @Access

```
@Entity
@Access(AccessType.FIELD)
public class Student {
    ...
    private String firstName;
    private String lastName;
    @Transient
     private Date dateOfBirth;
    ...
    @Access(AccessType.PROPERTY)
    @Column(name = “age", length = 3)
    public String getAge() {
        LocalDate date = dateOfBirth.toInstant().atZone(ZoneId.systemDefault()).toLocalDate();
        return Period.between(date, LocalDate.now()).getYears();
    }
}
```

ORM – Встраиваемые объекты

Аннотации @Embeddable и @Embedded

```
@Embeddable
public class Address {
    private String street;
    private String city;
    private String region;
    private String zipcode;
    private String country;
}

@Entity
public class Student {
    ...
    private String firstName;
    private String lastName;
    @Embedded
     private Address address;
    ...
}
```

ORM – Отображение связей – 1

В ООП связи между объектами классов определяются ассоциациями.

Свойства ассоциаций:

1. Направление: однонаправленная, двунаправленная.
2. Кардинальность: диапазон допустимого количества связей между объектами (от min до max): переменная или коллекция.

Однонаправленная ассоциация:

```
public class Car {
    private Color color;
}
public class Color {
    private int R;
    private int G;
    private int B;
}
```

Двунаправленная ассоциация:

```
public class Student {
    private Group group;
}
public class Group {
    private List<Student> students;
}
```

ORM – Отображение связей – 2

В РБД связи между сущностями определяются столбцами соединения или таблицами соединения с применением внешнего ключа.

Столбец соединения:

Car

Id	Color
1	11
2	12

Color

Id	R	G	B
11	255	255	255
12	128	128	128

Таблица соединения (один ко многим, многие ко многим):

Group

Id	Name
1	ПИбд-21
2	ПИбд-22

Student

Id	Name
11	Иванов
12	Петров
13	Сидоров

IdGroup	IdStudent
1	11
1	12
2	13

ORM – Отображение связей – 3

JPA позволяет отображать ассоциации в РБД.

JPA поддерживает:

Тип связи	Направление ассоциации	Аннотация
Один к одному	Однонаправленная	@OneToOne
Один к одному	Двунаправленная	@OneToOne
Один ко многим	Однонаправленная	@OneToMany
Многие к одному / один ко многим	Двунаправленная	@ManyToOne / @OneToMany
Многие к одному	Однонаправленная	@ManyToOne
Многие ко многим	Однонаправленная	@ManyToMany
Многие ко многим	Двунаправленная	@ManyToMany

ORM – Однонаправленная OneToOne 1

```
public class Student {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    private Address address;
}
public class Address {
    @Id
    @GeneratedValue
    private Long id;
    private String country;
    private String city;
}
```

id	name	address_id



id	country	city

ORM – Однонаправленная OneToOne 2

```
public class Student {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "add_fk", nullable = false)
    private Address address;
}
public class Address {
    @Id
    @GeneratedValue
    private Long id;
    private String country;
    private String city;
}
```

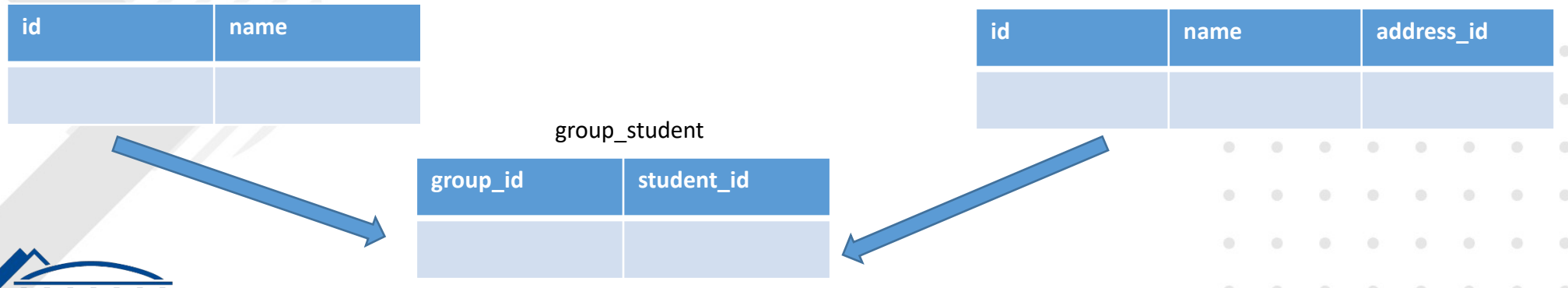
id	name	add_fk



id	country	city

ORM – Однонаправленная OneToMany 1

```
public class Group {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    private List<Student> students;
}
public class Student {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    private Address address;
}
```



ORM – Однонаправленная OneToMany 2

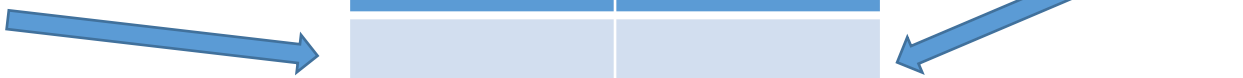
```
public class Group {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    @OneToMany
    @JoinTable(name = "students_in_groups",
        joinColumns = @JoinColumn(name = "group_fk"),
        inverseJoinColumns = @JoinColumn(name = "student_fk"))
    private List<Student> students;
}
public class Student {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    private Address address;
}
```

id	name

students_in_groups

group_fk	student_fk

id	name	address_id



ORM – Однонаправленная OneToMany 3

```
public class Group {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    @OneToMany
    @JoinColumn(name = "group_fk")
    private List<Student> students;
}
public class Student {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    private Address address;
}
```

id	name



id	name	address_id	group_fk

ORM – Двунаправленная ManyToMany 1

```
public class Group {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    @ManyToMany(mappedBy = "groups")
    private List<Student> students;
}
public class Student {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    private Address address;
    @ManyToMany
    @JoinTable(name = "students_groups",
        joinColumns = @JoinColumn(name = "student_fk"),
        inverseJoinColumns = @JoinColumn(name = "group_fk"))
    private List<Group> groups;
}
```

students_groups

id	name

group_fk	student_fk

id	name	address_id

ORM – Двунаправленная ManyToMany 2

```
public class Group {
    private List<Student> students;
    ...
    public void addStudent(Student student) {
        this.students.add(student);
        if (!student.getGroups().contains(this)) { // O(n) !!!
            student.getGroups().add(this);
        }
    }
}
...
public class Student {
    private List<Group> groups;
    ...
   public void addGroup(Group group) {
        this.groups.add(group);
        if (!group.getStudents().contains(this)) { // O(n) !!!
            group.getStudents().add(this);
        }
    }
}
```

ORM – Двунаправленная OneToMany

```
public class Group {
    private List<Student> students;
    ...
    public void addStudent(Student student) {
        this.students.add(student);
        if (student.getGroup() != this) {
            student.setGroup(this);
        }
    }
}
...
public class Student {
    private Group group;
    ...
   public void addGroup(Group group) {
        this.group = group;
        if (!group.getStudents().contains(this)) { // O(n) !!!
            group.getStudents().add(this);
        }
   }
}
```

ORM – Выборка связей

Аннотация	Стратегия выборки по умолчанию
@OneToOne	EAGER
@ManyToOne	EAGER
@OneToMany	LAZY
@ManyToMany	LAZY

ORM – Упорядочивание связей 1

```
public class Group {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    @OneToMany
    @OrderBy("name ASC")
    private List<Student> students;
}
public class Student {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    private Address address;
}
```

ORM – Упорядочивание связей 2

```
public class Group {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    @OneToMany
    @OrderColumn(name = "student_index")
    private List<Student> students;
}
public class Student {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    private Address address;
}
```

group_student

group_id	student_id	student_index

ORM – Отображение наследования 1

```
public class Person{
    @Id
    @GeneratedValue
    private Long id;
    private String name;
}
public class Student extends Person {
    private String document;
}
public class Teacher extends Person {
    private String position;
}
```

id	dtype	name	document	position

dtype (varchar, string) – столбец дискриминатора для определения типа сущности: физ. лицо, студент, преподаватель.

Возможные значения dtype: Person, Student, Teacher.

ORM – Отображение наследования 2

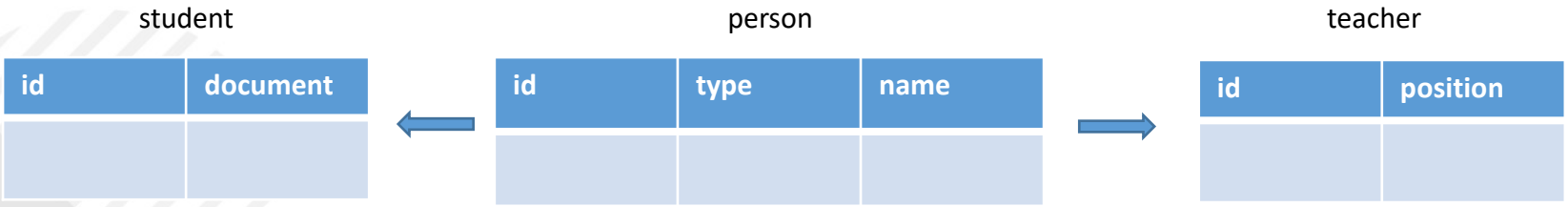
```
@DiscriminatorColumn (name="type",
   discriminatorType = DiscriminatorType.CHAR)
@DiscriminatorValue("P")
public class Person{
   @Id
   @GeneratedValue
   private Long id;
   private String name;
}
@DiscriminatorValue("S")
public class Student extends Person {
   private String document;
}
@DiscriminatorValue("T")
public class Teacher extends Person {
   private String position;
}
```

id	type	name	document	position

Возможные значения type: P, S, T.

ORM – Отображение наследования 3

```
@Inheritance(strategy = InheritanceType.JOINED)
public class Person{
    @Id
    @GeneratedValue
    private Long id;
    private String name;
}
public class Student extends Person {
    private String document;
}
public class Teacher extends Person {
    private String position;
}
```



Все связанные записи таблиц имеют одинаковый первичный ключ.

ORM – Отображение наследования 4

```
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public class Person{
    @Id
    @GeneratedValue
    private Long id;
    private String name;
}
public class Student extends Person {
    private String document;
}
public class Teacher extends Person {
    private String position;
}
```

student

id	name	document

person

id	name

teacher

id	name	position

Общий первичный ключ на все таблицы.

ORM – Отображение наследования 5

```
@MappedSuperclass
public class Person{
    private String name;
}
@Entity
public class Student extends Person {
    @Id
    @GeneratedValue
    private Long id;
    private String document;
}
@Entity
public class Teacher extends Person {
    @Id
    @GeneratedValue
    private Long id;
    private String position;
}
```

student

id	name	document

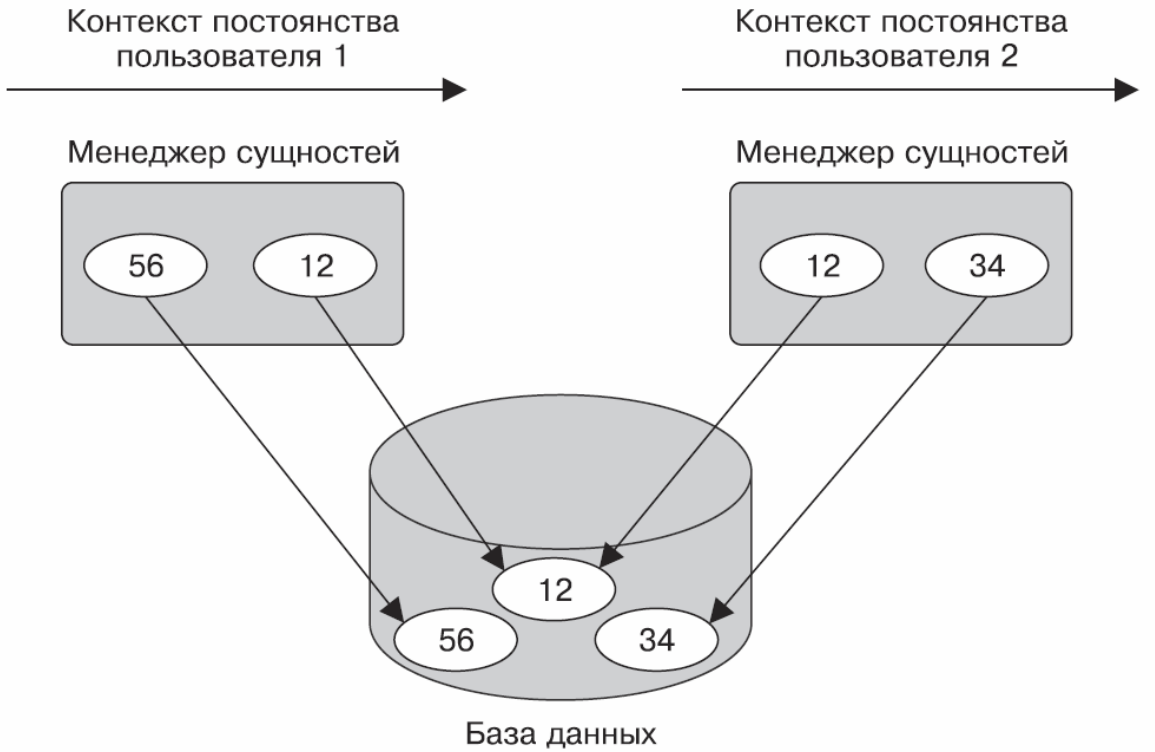
teacher

id	name	position

Менеджер сущностей

Менеджер сущностей (Entity Manager) – основной механизм JPA для обеспечения персистентности сущностей.

```
@PersistenceContext
private EntityManager em;
```



Основные методы менеджера сущностей

void persist(Object entity) – добавление сущности;
<T> T find(Class<T> entityClass, Object primaryKey) – поиск сущности;
void remove(Object entity) – удаление сущности;
<T> T merge(T entity) – обновление сущности;
void refresh(Object entity) – обновление сущности в контексте данными из БД;
void flush() – синхронизация контекста с БД;
void clear() – очищение контекста;
void detach(Object entity) – удаление сущности из контекста;
boolean contains(Object entity) – проверка нахождения сущности в контексте.

Управление транзакциями

Аннотация @Transactional используется для облегчения работы с транзакциями.

@Transactional

аналогично

```
Transaction transaction = entityManager.getTransaction()
try {
   transaction.begin();
   em.persist(ent1);
   em.flush();
   em.persist(ent2);
   transaction.commit();
}
catch(Exception ex) {
   transaction.rollback();
   throw ex;
}
```

Удаление сущностей-сирот

```
public class Student {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    @OneToOne (orphanRemoval=true)
    private Address address;
}
public class Address {
    @Id
    @GeneratedValue
    private Long id;
    private String country;
    private String city;
}
```

Каскадирование событий

У аннотаций @OneToOne, @OneToMany, @ManyToOne и @ManyToMany есть атрибут cascade, который принимает массив событий для каскадирования.

CascadeType	Описание
PERSIST	Каскадирует операции по обеспечению постоянства
REMOVE	Каскадирует операции удаления
MERGE	Каскадирует операции объединения
REFRESH	Каскадирует операции обновления
DETACH	Каскадирует операции отсоединения
ALL	Каскадирует все операции

SQL (нативные) запросы

SELECT s FROM Student s WHERE s.lastName = 'Иванов’

```
List<Student> students = em
    .createNativeQuery("SELECT * FROM student AS s
                WHERE s.last_name='Иванов'", Student.class)
    .getResultList();
```

```
@Entity
@NamedNativeQuery(name = "findAll", query="select * from student")
public class Student {
    ...
}
```

JPQL

JQPL (Java Persistence Query Language) – объектно-ориентированный язык запросов очень похожий на язык SQL.

```
SELECT COUNT(s) FROM Student s WHERE s.name = :name GROUP BY s.group

SELECT s.document FROM Student s WHERE s.name = 'Иван'
    AND s.age BETWEEN 18 AND 20
    AND (s.address.city NOT IN ('Ульяновск', 'Димитровград')
    OR s.email LIKE '%ulstu.ru‘)

SELECT p.name, CASE p.type WHEN 'S' THEN 'Student' ELSE 'TEACHER'
    FROM Person p ORDER BY p.name DESC

SELECT DISTINCT s.address.country FROM Student s

SELECT NEW ru.ulstu.is.Student(p.name) FROM Person p

DELETE FROM Student s

UPDATE Student s SET s.document = '' WHERE s.age > 30
```

Динамические запросы

Динамические запросы представляют собой запросы на языке JPQL.

```
List<Student> students = em.createQuery("SELECT s FROM Student s", Student.class)
    .getResultList();

List<Student> students = em.createQuery("SELECT s FROM Student s WHERE
s.firstName=:fname", Student.class)
    .setParameter("fname", «Иван")
    .getResultList();

Student student = em.createQuery("SELECT s FROM Student s WHERE s.id=:id",
Student.class)
    .setParameter("id", 100)
    .getSingleResult();

em.createQuery("DELETE FROM Student").executeUpdate();
```

Именованные запросы

Именованные запросы – определяются для сущностей и преобразуются в SQL один раз при запуске программы, а не при каждом вызове.

```
@Entity
@NamedQueries({
   @NamedQuery(name = Student.FIND_ALL, query="select s from Student s"),
   @NamedQuery(name = Student.FIND_BY_ID, query="select s from Student s where s.id = :id"),
   @NamedQuery(name = Student.FIND_BY_LNAME, query="select s from Student s
                where s.lastName = :lname")
})
public class Student {
   public static final String FIND_ALL = "Student.findAll";
   public static final String FIND_BY_ID = "Student.findById";
   public static final String FIND_BY_LNAME = "Student.findByLastName";
   ...
}

List<Student> students = em.createNamedQuery(Student.FIND_BY_LNAME)
   .setParameter("lname", "Иванов")
   .setMaxResults(3)
   .getResultList();
```

Criteria API

Criteria API – объектно-ориентированный механизм формирования запросов с помощью операторов.

SELECT s FROM Student s WHERE s.lastName = 'Иванов’

```
CriteriaBuilder builder = em.getCriteriaBuilder();
CriteriaQuery<Student> criteriaQuery = builder.createQuery(Student.class);
Root<Student> root = criteriaQuery.from(Student.class);
criteriaQuery.select(root).where(builder.equal(root.get("lastName"), "Иванов"));
List<Student> students = em.createQuery(criteriaQuery).getResultList();
```

Репозитории (DAO)

```
public interface SomeRepository extends CrudRepository<SomeEntity, Long> {
}
```

Интерфейс CrudRepository определяет перечень методов для:

- создания сущностей;
- поиска сущностей;
- изменения сущностей;
- удаления сущностей;
- создания запросов к БД (native SQL, JPQL);
- создания декларативных запросов к БД (название метода определяет запрос к БД).

```
@Query("SELECT u FROM User u WHERE u.status = 1")
Collection<User> findAllActiveUsers();

@Query(
  value = "SELECT * FROM USERS u WHERE u.status = 1",
  nativeQuery = true)
Collection<User> findAllActiveUsersNative();

List<User> findByNameOrBirthDateAndActive(String name, ZonedDateTime birthDate, Boolean active);
```

Конкурентный доступ – 1

Состояние гонки – может возникнут в БД, когда несколько транзакций выполняются одновременно над одними данными.

```
tx1.begin();

// Цена книги равна $10
Book book = em.find(Book.class, 12);

book.raisePriceByTwoDollars();

tx1.commit();
// Теперь цена равна $12
```

Время

```
tx2.begin();

// Цена книги равна $10
Book book = em.find(Book.class, 12);

book.raisePriceByFiveDollars();

tx2.commit();
// Теперь цена равна $15
```

Конкурентный доступ – 2

Блокировки – решение проблемы синхронного доступа к критической области.

Типы блокировок:

1. Оптимистическая блокировка основана на предположении, согласно которому большинство транзакций в базах данных не конфликтует с другими транзакциями, обеспечивая как можно более свободный конкурентный доступ, когда разрешается выполнение транзакций.
2. Пессимистическая блокировка основана на противоположном предположении, в результате блокировка будет применяться к ресурсу до начала операций с ним.

Конкурентный доступ – 3

Методы менеджера сущностей:

1. <T> T find(Class<T> entityClass, Object primaryKey, LockModeType lockMode) – выполняет поиск сущности указанного класса и первичного ключа, а затем блокирует ее согласно заданному типу режима блокировки;
2. void lock(Object entity, LockModeType lockMode) – блокирует экземпляр сущности, который содержится в контексте постоянства, согласно заданному типу режима блокировки;
3. void refresh(Object entity, LockModeType lockMode) – обновляет состояние экземпляра из базы данных, перезаписывая изменения, внесенные в сущность, при наличии таковых, и блокирует ее согласно заданному типу режима блокировки;
4. LockModeType getLockMode(Object entity) – извлекает значение текущего режима блокировки для экземпляра сущности.

Методы для блокировки JPQL-запросов:

1. LockModeType getLockMode() – извлекает значение текущего режима блокировки для запроса;
2. Query setLockMode(LockModeType lockMode) – задает тип режима блокировки для использования при выполнении запроса.

Spring Data JPS

@Lock(LockModeType.OPTIMISTIC_FORCE_INCREMENT)

Конкурентный доступ – 4

Возможные значения LockModeType:

- OPTIMISTIC – применяет оптимистическую блокировку;
- OPTIMISTIC_FORCE_INCREMENT – применяет оптимистическую блокировку и форсирует инкрементирование значения столбца version, связанного с сущностью;
- PESSIMISTIC_READ – применяет пессимистическую блокировку без необходимости в повторном чтении данных в конце транзакции для обеспечения блокировки;
- PESSIMISTIC_WRITE – применяет пессимистическую блокировку и форсирует сериализацию между транзакциями, которые пытаются обновить сущность;
- PESSIMISTIC_FORCE_INCREMENT – применяет пессимистическую блокировку и форсирует инкрементирование значения столбца version, связанного с сущностью;
- NONE – определяет, что не должен использоваться никакой механизмблокировки.

Конкурентный доступ – 5

Чтение, а затем блокировка:

```
Book book = em.find(Book.class, 12);
em.lock(book, LockModeType.OPTIMISTIC_FORCE_INCREMENT);
book.raisePriceByTwoDollars();
```

Чтение и блокировка:

```
Book book = em.find(Book.class, 12, LockModeType.OPTIMISTIC_FORCE_INCREMENT);
book.raisePriceByTwoDollars();
```

Контроль версий – 1

Версионирование позволяет отследить изменения значений атрибутов сущности.

При каждом изменении значения атрибута сущности счетчик версии увеличивается на 1.

```
public class Student {
    @Id
    @GeneratedValue
    private Long id;
    private String firstName;
    private String lastName;
    @Version
    private Integer version;
}

Student student = new Student(...);
em.persist(student); // student.version == 1
student.setLastName(...);
em.merge(student); // student.version == 2
```

Контроль версий – Оптимистичная блокировка

```
tx1.begin();

// Цена книги равна $10
Book book = em.find(Book.class, 12);
// book.getVersion() == 1

book.raisePriceByTwoDollars();

tx1.commit();
// Теперь цена равна $12
// book.getVersion() == 2
```

Время

```
tx2.begin();

// Цена книги равна $10
Book book = em.find(Book.class, 12);
// book.getVersion() == 1

book.raisePriceByFiveDollars();

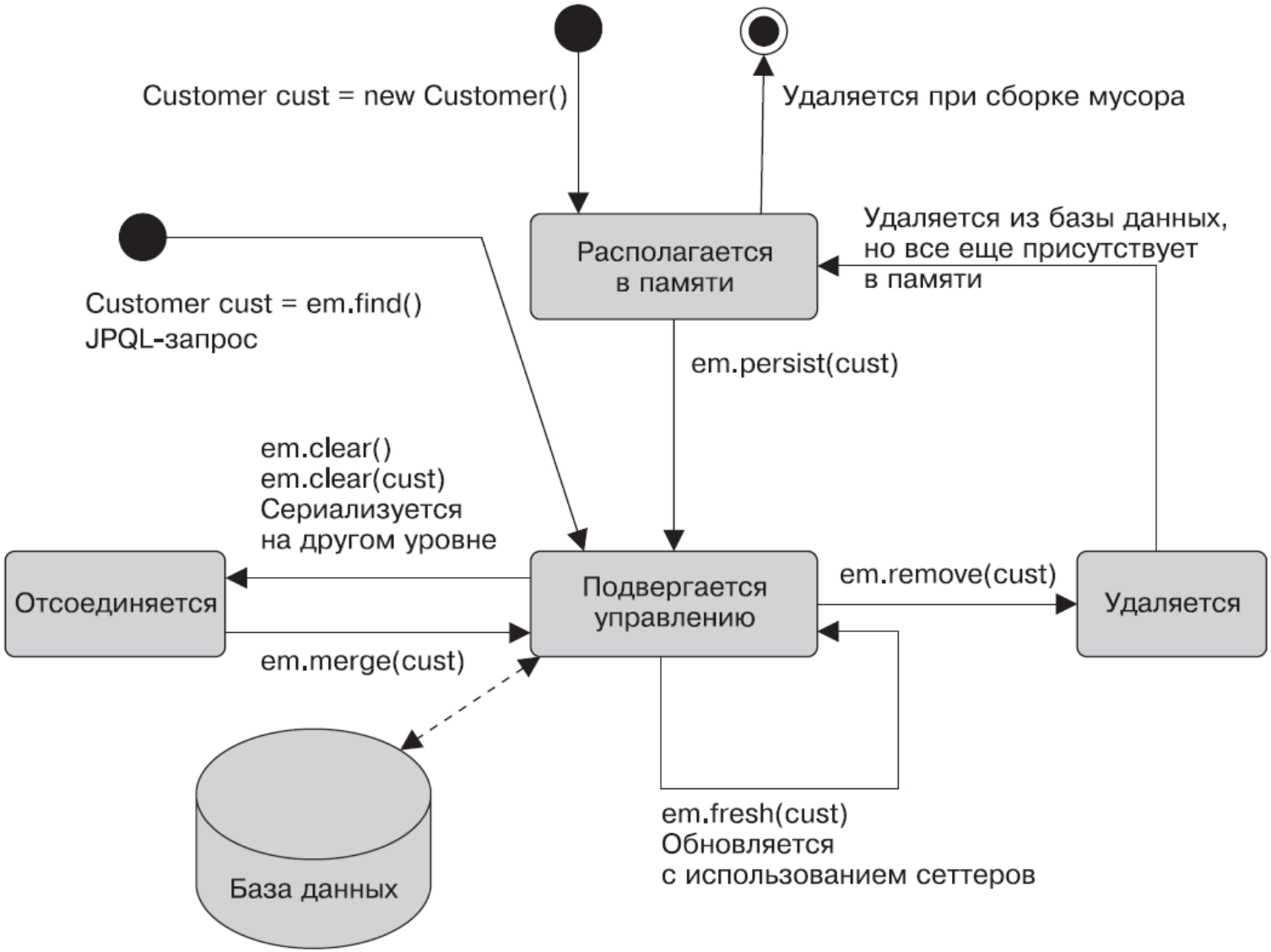
tx2.commit();
// Номер версии должен равняться 1,
// но он равен 2
// OptimisticLockException
```

Контроль версий – Пессимистичная блокировка

При использовании пессимистичной блокировки строка таблицы блокируется полностью до завершения транзакции.

Пессимистичная блокировка реализуется средствами СУБД.

Жизненный цикл сущности – 1

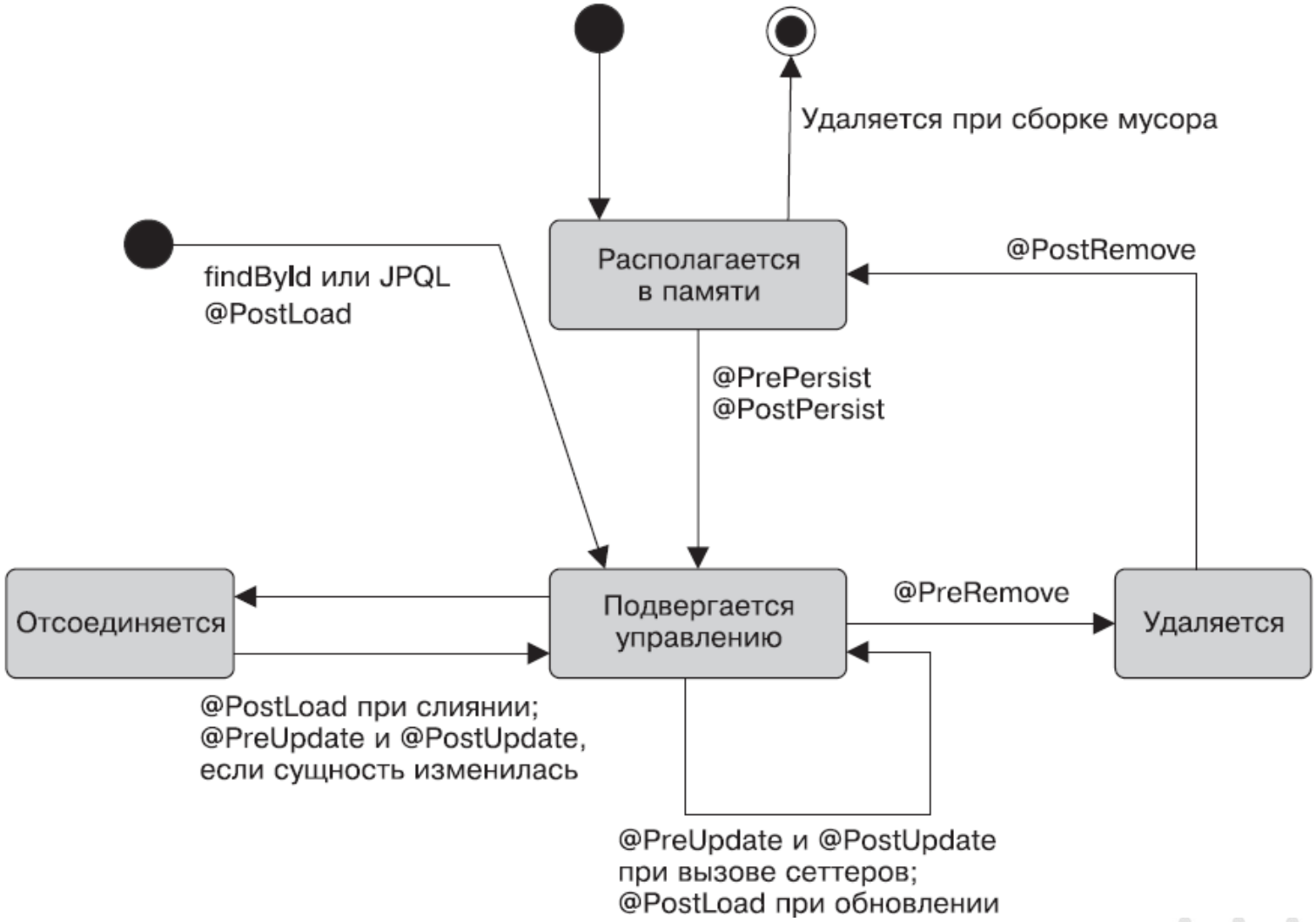


Жизненный цикл сущности – 2

Аннотации для перехвата событий жизненного цикла:

- @PrePersist – до выполнения EntityManager.persist();
- @PostPersist – после выполнения EntityManager.persist();
- @PreUpdate – до выполнения EntityManager.merge() или вызова сеттеров;
- @PostUpdate – после обновления сущности;
- @PreRemove – до выполнения EntityManager.remove();
- @PostRemove – после выполнения EntityManager.remove();
- @Postload – после загрузки данных сущности.

Жизненный цикл сущности – 3



Жизненный цикл сущности – 4

```
public class Student {
    …

    @PrePersist
    @PreUpdate
    private void validate() {
        if (firstName == null || "".equals(firstName))
            throw new IllegalArgumentException("Неверное имя");
        if (lastName == null || "".equals(lastName))
            throw new IllegalArgumentException("Неверная фамилия");
    }
}
```

Жизненный цикл сущности – 5

```
public class StudentAgeListener {
   @PostLoad
   @PostPersist
   @PostUpdate
   private void calculateAge(Student student) {
      LocalDate date = student.dateOfBirth.toInstant().atZone(ZoneId.systemDefault()).toLocalDate();
      student.setAge(Period.between(date, LocalDate.now()).getYears());
   }
}

public class StudentValidationListener {
   @PrePersist
   @PreUpdate
   private void validate(Customer customer) {
      if (customer.getFirstName() == null || "".equals(customer.getFirstName()))
         throw new IllegalArgumentException("Неверное имя");
      if (customer.getLastName() == null || "".equals(customer.getLastName()))
         throw new IllegalArgumentException("Неверная фамилия");
   }
}

@EntityListeners({StudentAgeListener.class, StudentValidationListener.class})
@Entity
public class Student {
```

Спасибо за внимание!

Литература:

1. Изучаем Java EE 7. – СПб.: Питер, 2014. – 640 с.
2. Spring 4 для профессионалов, 4-е изд. : Пер. с англ. – М. : ООО «И.Д. Вильямс», 2015. – 752 с.
3. Spring Data JPA - Reference Documentation. URL: https://docs.spring.io/spring-data/jpa/docs/current/reference/html/index.html
4. Java Persistence. URL: https://en.wikibooks.org/wiki/Java_Persistence
5. Односторонние и двусторонние связи https://www.baeldung.com/jpa-hibernate-associations
6. Getters и Setters для двусторонних связей https://en.wikibooks.org/wiki/Java_Persistence/OneToMany - Getters_and_Setters
7. Многие-ко-многим с доп. атрибутами https://thorben-janssen.com/hibernate-tip-many-to-many-association-with-additional-attributes/

Ульяновский государственный технический университет

2022